

DELTA LAKE MERGE IMPLEMENTATION USING PANDAS AND PYSPARK

Assignment Report

Name: Eklavya

Assignment Title: Week 7 - Delta Lake MERGE Implementation

Platform Used: Databricks Free Edition

Programming Language: Python

Libraries & Technologies: Pandas, PySpark, Delta Lake

Table of Contents

1. Introduction
2. Objective
3. Dataset Description
4. Tools and Technologies Used
5. Part A – Data Cleaning using Pandas
6. Part B – Delta Lake MERGE Implementation
7. Validation and Results
8. Challenges Faced
9. Learning Outcomes
10. Conclusion
11. References

1. Introduction

Data preprocessing and incremental data processing are fundamental tasks in data engineering. Before any meaningful analysis can be performed, raw datasets must be cleaned, validated, and transformed into a structured format. Pandas is one of the most widely used Python libraries for data manipulation and analysis, while Delta Lake provides reliable storage with ACID transactions and supports efficient incremental updates through the MERGE operation.

In this assignment, the Sample Superstore dataset was first cleaned using Pandas. The cleaned dataset was then loaded into Apache Spark and stored as a Delta table. An incremental dataset was created to simulate new business transactions, and the Delta Lake MERGE operation was used to update existing records and insert new records into the master table.

The assignment demonstrates a complete data engineering workflow, starting from data cleaning to incremental processing using modern big data technologies.

2. Objective

The primary objectives of this assignment were:

- Load a CSV dataset into a Pandas DataFrame.
 - Explore the dataset using different Pandas functions.
 - Identify and handle missing values.
 - Remove duplicate records.
 - Perform filtering and column selection.
 - Create derived columns.
 - Save the cleaned dataset as a CSV file.
 - Load the cleaned dataset into Apache Spark.
 - Create a Delta Lake master table.
 - Generate an incremental dataset.
 - Apply the Delta Lake MERGE operation.
 - Validate the final dataset after the MERGE process.
-

3. Dataset Description

The dataset used in this assignment is the **Sample Superstore Dataset**, obtained from Kaggle.

The dataset contains sales information of a retail superstore and includes details such as:

- Order ID
- Order Date

- Customer Information
- Product Details
- Sales
- Quantity
- Discount
- Profit
- Region
- Category

The original dataset consists of **9,994 records** and **21 columns**. During the Pandas implementation, two additional derived columns were created, increasing the total number of columns.

4. Tools and Technologies Used

The following tools and technologies were used throughout the assignment:

- Python
- Pandas
- PySpark
- Delta Lake
- Apache Spark
- Databricks Free Edition
- GitHub

These technologies enabled efficient data cleaning, transformation, storage, and incremental processing.

5. Part A – Data Cleaning using Pandas

Step 1: Loading the Dataset

The Sample Superstore dataset was loaded into a Pandas DataFrame using the `read_csv()` function. This allowed the dataset to be processed efficiently using Pandas operations.

Step 2: Data Exploration

The dataset was explored using several Pandas functions to understand its structure and contents.

The following operations were performed:

- Displayed the first five rows using `head()`
- Displayed the last five rows using `tail()`
- Checked the dataset dimensions using `shape`
- Displayed all column names
- Examined the data types of each column
- Generated dataset information using `info()`

These exploratory steps helped in understanding the dataset before applying transformations.

Step 3: Handling Missing Values

Missing values were identified using:

- `isnull()`
- `sum()`

The dataset was examined carefully to determine whether any missing values existed. Since the dataset contained no significant missing values, no additional imputation techniques were required.

Step 4: Removing Duplicate Records

Duplicate records reduce data quality and may lead to incorrect analysis.

The following operations were performed:

- Checked duplicate rows using `duplicated()`
- Removed duplicate rows using `drop_duplicates()`
- Verified that no duplicate records remained

This improved the quality and consistency of the dataset.

Step 5: Filtering and Selecting Data

Basic Pandas operations were performed, including:

- Selecting specific columns
- Filtering rows based on conditions

These operations demonstrated the ability to manipulate and retrieve relevant information from the dataset.

Step 6: Creating Derived Columns

Two new columns were created.

Price

The price per unit was calculated using the following formula:

$$\text{Price} = \text{Sales} \div \text{Quantity}$$

This represents the average selling price of a single unit.

Total Amount

A second derived column named **total_amount** was created using:

$$\text{total_amount} = \text{Price} \times \text{Quantity}$$

This value represents the total transaction amount and matches the original sales amount.

Step 7: Saving the Cleaned Dataset

After completing all cleaning operations, the processed dataset was exported as:

cleaned_superstore.csv

This cleaned dataset was used in the Delta Lake implementation.

6. Part B – Delta Lake MERGE Implementation

Step 1: Loading the Cleaned Dataset

The cleaned CSV file was loaded into Apache Spark using PySpark.

The schema was verified to ensure that all columns had the correct data types before storing the data in Delta Lake.

Step 2: Renaming Columns

Delta Lake does not allow spaces or certain special characters in column names by default.

Therefore, all spaces and hyphens were replaced with underscores.

For example:

- Row ID → Row_ID
- Customer Name → Customer_Name
- Product Name → Product_Name
- Sub-Category → Sub_Category

This ensured compatibility with Delta Lake.

Step 3: Creating the Master Delta Table

The cleaned Spark DataFrame was stored as a Delta table named:

customer_master

This table acted as the primary dataset for incremental processing.

Step 4: Creating Incremental Data

To simulate new business transactions, an incremental dataset was created.

The incremental dataset contained:

- Five updated records
- Five newly inserted records

The updated records had modified Sales and Profit values, while the new records were assigned new Row_ID values.

The combined dataset was stored as:

customer_incremental

Step 5: Delta Lake MERGE Operation

The MERGE operation was performed using the customer_incremental table as the source and customer_master as the target.

The MERGE operation performed two actions:

Update

Existing records with matching Row_ID values were updated automatically.

Insert

Records with new Row_ID values were inserted into the master table.

This demonstrated incremental data loading using Delta Lake.

7. Validation and Results

After completing the MERGE operation, several validation checks were performed.

Record Count Validation

Original Records:

9994

Incremental Records:

10

Final Records:

9999

This confirmed that five existing records were updated while five new records were inserted.

Duplicate Validation

The Row_ID column was checked for duplicate records.

No duplicate Row_ID values were found after the MERGE operation.

Updated Records

The updated records were verified successfully.

The Sales and Profit values reflected the expected changes introduced during incremental processing.

Newly Inserted Records

The newly inserted records were identified using:

Row_ID > 100000

These records confirmed that the MERGE operation inserted new records successfully.

8. Challenges Faced

During the implementation of the assignment, several practical challenges were encountered.

Initially, the CSV file was incorrectly interpreted while being read into Spark, causing column misalignment. This issue was resolved by using the correct Spark CSV reading options and validating the schema.

Another challenge was that Delta Lake does not allow spaces in column names by default. The problem was solved by renaming all column names using underscores before creating the Delta table.

Several data type conversion issues also occurred while creating the Delta table. These were resolved by validating the schema and recreating the Delta table after correcting the data loading process.

Working with Databricks Free Edition Serverless required careful handling of Spark operations and Delta table creation, which provided valuable practical experience with cloud-based data engineering tools.

9. Learning Outcomes

Through this assignment, the following concepts were learned:

- Data exploration using Pandas
- Data cleaning techniques
- Missing value analysis
- Duplicate removal
- Derived column creation
- CSV export
- Spark DataFrame operations
- Delta Lake table creation
- Incremental data processing
- Delta Lake MERGE operation
- Data validation techniques
- GitHub project organization

The assignment also provided practical experience with Databricks and Delta Lake.

10. Conclusion

This assignment successfully demonstrated an end-to-end data engineering workflow using Pandas, Apache Spark, and Delta Lake.

The dataset was first cleaned using Pandas, duplicate records were removed, missing values were analyzed, and new derived columns were created. The cleaned dataset was then loaded into Delta Lake, where a master table and an incremental dataset were created.

The Delta Lake MERGE operation successfully updated existing records and inserted new records into the master table. Finally, validation steps confirmed that the data was accurate, consistent, and free from duplicate Row_ID values.

Overall, this assignment provided practical knowledge of modern data engineering concepts and demonstrated the use of Delta Lake for efficient incremental data processing.

11. References

1. Kaggle – Sample Superstore Dataset

<https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

1. Microsoft Learn – Delta Lake MERGE Documentation

<https://learn.microsoft.com/en-us/azure/databricks/delta/merge>